Lastseat: Design

How Lastseat stores teams, why two people can never take the same seat, and what I traded away to get there.

Data model

MongoDB, ten collections. Three of them matter:

- **`teams`**: one document per team with its members embedded (at most `maxSize`), plus `memberCount`, `openSeats`, `leaderId` and a `version` that increases on every change. A team is small and always read whole, and one document is the unit MongoDB updates atomically.
- **`participants`**: one row per person per hackathon: skills, time zone, `lookingForTeam` and a copy of `teamId`. It is the "people looking" board and the input to suggestions.
- **`audit_log`**: append-only: who, what, when, and a snapshot of the team after the change. It is also the event stream for live updates.

Around them: `invites` (SHA-256 of the code, `expiresAt`, `revokedAt`), `join_requests`, `idempotency_keys`, `rate_limits`, `sessions`, `users`, `hackathons`.

How races are prevented

The last seat is taken by one conditional update: `findOneAndUpdate({ _id, openSeats: { $gt: 0 }, 'members.userId': { $ne: me } }, { $push, $inc })`. The check is inside the write. MongoDB applies writes to one document one at a time, so of 50 simultaneous joins exactly one matches; the other 49 match nothing and get `TEAM_FULL`.

One team per person is a unique multikey index on `(hackathonId, members.userId)`. A second team containing the same person cannot be written, by a join, a create or a bug.

Everything else a join touches (the participant row, the person's other requests, the audit entry, the idempotency record) commits in the same transaction as the claim. Losers get a write conflict, roll back and run again. They first wait about as long as the failed attempt took, so each needs one retry whether the database is fast or rate-limited.

Retries. A write may carry an `Idempotency-Key`; its result is stored in that same transaction, so a double click replays the first response. Joining a team you are already in is a no-op even without a key.

Authorization and limits. "Only the leader" is part of the write filter (`leaderId: actor`), not a check before it. Leave and remove write with the `version` they read. Counted limits (5 pending requests, 5 active codes) first write one shared document, so concurrent attempts conflict and run one after another.

Proof. `race.test.js` fires real concurrent HTTP requests: 50 and then 300 for one seat, 40 for three, 10 parallel accepts, one person into two teams, 20 simultaneous creates, the last member leaving against 20 joins, and a mixed storm. After each, an invariant checker reads the whole database. `chaos.test.js` kills the server between the claim and the commit. The live site has a button that runs the 50-way race on the production database.

Trade-offs

Decision	What I gave up
Transactions for the bookkeeping	Needs a replica set. A server killed mid-transaction leaves that team write-locked until MongoDB times it out (60 s default; I measured 77 s). Bounded: after 8 s a request gets a retryable `503 BUSY`.
Audit log is also the event bus (no Redis)	Events reach every instance, and only after commit; but log retention and delivery are coupled, and each instance holds a change stream.
SSE instead of WebSockets	One direction is all the board needs and reconnecting is built in. A reconnecting client refetches rather than replaying missed events.
Board pages cached, emptied by the same events	Exact on one instance. Across instances a page can lag by the skew between their change streams, capped by a 2 s lifetime.
Sessions trusted for 10 s	A logout takes up to 10 s to reach other instances. In return a board read makes no session query.
Member names and skills copied into the team	Board reads need no join; a profile edit updates two documents, in one transaction.
Invite codes stored only as a hash	60 bits and 10 guesses per 10 minutes per account: about a billion years to hit one of 2,000 live codes. The leader sees a code once and must create a new one if it is lost.
Minimum size is a status, not a gate	A team below the minimum exists as "forming" and cannot compete yet. Refusing it would make starting a team impossible.

With one more week

1. Remove the 60 s lock after a crash: commit the seat with the single-document write alone and move the bookkeeping to an outbox inside the team document, drained by a relay.
2. Replay missed events on reconnect using `Last-Event-ID` and the audit log.
3. A nightly soak of the race tests, and load tests against several instances and a database tier without a rate limit.
4. Organizer tools: change size limits, lock teams at the deadline, export.
5. A shared store for the per-request rate limit, which is per instance today.